ZOKA QUANTIC

Post-Quantum Privacy by Default — Version 2.0 (Draft)

Status: Draft design, October 2026. Zoka Quantic is not live. The network that runs today, Mainnet Zenith, becomes the Zoka testnet, and every new component described here is tested there before it reaches the Zoka Quantic genesis block.

Status labels used throughout:

Label	Meaning
Live	Runs on today's network (Zenith).
In development	Being built; it will run on the testnet before genesis.
Research	The design is not settled yet.
Planned	Committed, not started.
Removed	Shipped before, no longer part of ZOKA.

Abstract

A public ledger is permanent. Anyone can store it today and analyse it with whatever computers exist in fifteen years. For a privacy chain, a future quantum computer is therefore not a future problem: what it will be able to read is being written now.

Zoka Quantic is the second generation of ZOKA and a new chain. Its rule is simple: **everything that protects privacy is post-quantum from the genesis block** — who received a payment, how much, and which note each transaction spent. Everything that protects validity — that nobody can create coins or spend what is not theirs — stays on classical cryptography for now.

The asymmetry is deliberate. A break of validity can only be exploited after a large quantum computer exists, and it can be answered then with a protocol upgrade. A break of privacy reaches backwards over every transaction already on the chain, and nothing can undo it.

Zoka Quantic is **not** "quantum-proof", and this document never claims it is. It is a private chain whose privacy is designed to survive a quantum computer, and whose validity has a stated path for when one appears.

The design combines:

1. **Hybrid stealth addresses**: elliptic-curve key agreement combined with ML-KEM-768 (NIST FIPS 203). Both must be broken to learn who received an output or how much.
2. **Spending by proof alone**: a Groth16 proof shows that the spent note exists and that its nullifier is correct, without publishing which note it is. No rings, no key images.
3. **Pedersen commitments and 64-bit Bulletproof range proofs** for confidential amounts.

4. **Hash-based nullifiers** derived from a key that is never published as a curve point.
5. **Hybrid Ed25519 + ML-DSA-65** (NIST FIPS 204) signatures on bootstrap snapshots.

Consensus is RandomX-compatible proof of work. The token is **ZKA**, capped at 23,000,000, of which 5,000,000 are allocated to the team at genesis (section 8).

---

1. Why a New Chain

Post-quantum privacy cannot be retrofitted. Every output on the Zenith chain was created with classical key agreement. Changing the rules from some block onward would leave every earlier output readable to a future quantum adversary, forever. A new genesis lets every output be born under the new rules.

Zenith publishes which note each transaction spends. In October 2026 we found that the Zenith private-transaction format carries, in the clear, the commitment and the one-time public key of every note it consumes. We confirmed it over the whole chain: its 254 private transactions publish 834 spent notes; only 254 of them sit inside a ring signature, one per transaction; and 559 can be linked directly to the output that created them. Amounts and recipients stay hidden. The transaction graph does not, and no quantum computer is needed to see it. The ring signature does not help, because the real input is published next to the ring. This is a design error of version 1, and section 4.6 is how Zoka Quantic removes it.

New formats are cleaner at genesis. Larger addresses, new output fields and a different spend proof are simpler to introduce on an empty ledger than as a fork of a live one.

Zenith keeps running as the Zoka testnet. Its coins have no value and **do not migrate**: no ZKA on Zenith was ever sold for money, so Zoka Quantic starts from an empty ledger.

---

2. Scope and Status

Component	Status
Private transactions: Pedersen commitments, Bulletproofs, Groth16 (BLS12-381)	Live
RandomX-compatible proof of work, 120-second target	Live
libp2p transport: TCP, Noise, Yamux, Gossipsub, Kademlia	Live
Tor onion seed published by the desktop wallet (managed Tor 0.4.9.13)	Live
Snapshot bootstrap over iroh (QUIC with hole punching); peers serve each other	Live
ZSilent Core desktop wallet (Windows, Linux)	Live
ZKAPriv Android wallet: peer-to-peer with embedded Tor (Arti)	Live

Team mining; explorer built into the wallet	Live
P2P Exchange (ZKA for USDT)	Removed, October 2026
Hybrid stealth addresses with ML-KEM-768	In development
Spending without publishing inputs, rings or key images	Research
Hybrid Ed25519 + ML-DSA-65 snapshot signatures	Planned
Multi-party trusted-setup ceremony for the final circuits	Planned, before genesis
External review of the post-quantum components	Planned, before genesis
Public seed servers, in addition to Tor	Planned
Zoka Quantic genesis	Planned

No date is committed for genesis. It happens when the items marked "before genesis" are done.

---

3. Threat Model

Zoka Quantic is designed to defend against:

1. A passive observer of the public chain, including anyone indexing block data.
2. **A future observer with a cryptographically relevant quantum computer** analysing the archived chain ("harvest now, decrypt later"). That observer must not learn who received an output, how much it carried, or which note a transaction spent.
3. A node operator who reads all gossiped traffic.
4. An adversary holding the encrypted wallet file without its password.

It does **not** defend against:

1. **Forgery once such a quantum computer exists.** Groth16, Bulletproofs and the binding property of Pedersen commitments rest on discrete-logarithm-type assumptions. A large quantum computer could forge proofs and create coins without detection, or spend outputs it does not own. This risk only runs forward in time. The response is a protocol upgrade to hash-based proofs before such machines exist, and every format carries a version identifier so that the upgrade can be a fork rather than another new chain.
2. A global passive adversary watching both ends of every network link in real time.
3. A user who reuses one receive address across separate off-chain identities.
4. A compromised spending key. There is no recovery backdoor.
5. Local disk compromise combined with knowledge of the wallet password.

---

4. Cryptographic Stack

4.1 Primitives

Primitive	Use	Against a quantum computer
ML-KEM-768 (FIPS 203)	Post-quantum half of output key agreement	Resists
Key agreement on Ristretto255	Classical half, kept so the hybrid is never weaker than today	Breaks; the hybrid still holds
Groth16 over BLS12-381	Spend and validity proofs	Proofs reveal nothing (perfect zero knowledge); soundness breaks
Pedersen commitments, Ristretto255	Confidential amounts	Hiding holds (perfectly hiding); binding breaks
Bulletproofs, 64-bit	Range proofs on committed amounts	Soundness breaks
BLAKE3, SHA-256, SHA-3	Nullifiers, key derivation, hashing	Resists
Ed25519 + ML-DSA-65 (FIPS 204)	Bootstrap snapshot signatures	Resists (ML-DSA half)
Argon2 + AES-256-GCM	Wallet file encryption	Resists
RandomX	Proof of work	Not affected in practice

Every line marked "breaks" concerns validity, not privacy, and is covered by item 1 of "does not defend against" above.

4.2 Pedersen Commitments

For a value `v` and a blinding factor `r`:

```
C(v, r) = v*G + r*H
```

with independent generators `G` and `H` over Ristretto255. Commitments are perfectly hiding, computationally binding and additively homomorphic.

4.3 Confidential Balance Equation

Every private transaction satisfies:

```
sum(C_in) = sum(C_out) + C_fee
```

over the group, with the blinding factors balanced by the sender. In Zoka Quantic the input commitments are not published (section 4.6), so this equation is checked inside the zero-knowledge proof rather than in the clear.

4.4 Range Proofs

Each committed amount is constrained to `0 <= v < 2^64` by a Bulletproof, which prevents overflow-based inflation while the amount stays hidden.

4.5 Hybrid Stealth Addresses — In development

A recipient holds three key pairs: a scan pair `(s, S = s*G)`, a spend pair `(b, B = b*G)` and an ML-KEM-768 pair `(dk, ek)`. The address encodes `S`, `B` and `ek` with a checksum.

For each output the sender:

1. picks a fresh `r` and publishes `R = r*G`;
2. computes the classical secret `k_ec = r*S`;
3. runs `(ct, k_pq) = ML-KEM.Encaps(ek)` and publishes `ct`;
4. combines both into one secret, `k = KDF(k_ec || k_pq || ct || R || S || ek)`, following an established hybrid construction (X-Wing) rather than an ad-hoc one;
5. derives from `k` the output's one-time key `P = H(k, "spend")*G + B`, a view tag, the key that encrypts the amount opening, and the key that encrypts the miner's fee claim.

The recipient recomputes `k_ec = s*R` and `k_pq = ML-KEM.Decaps(dk, ct)`, and from there the same `k`. An adversary who breaks discrete logarithms still lacks `k_pq`; one who breaks ML-KEM still lacks `k_ec`. Learning who received an output, or how much, requires breaking both.

Costs, stated up front:

- An address grows from 68 to about 1,250 bytes, roughly 1,700 characters in Base58. Wallets show it as a QR code.
- Each output carries a 1,088-byte ML-KEM ciphertext.
- Scanning costs one ML-KEM decapsulation per output that passes the view tag. It will be measured on a 32-bit Android phone before the parameter set is fixed; ML-KEM-512 remains the fallback if 768 proves too slow there.

4.6 Spending Without Revealing the Spent Note — Research

On Zenith a private transaction publishes, for each input, its commitment, its one-time public key, a key image and a ring of candidate keys (section 1 explains why that leaks the transaction graph).

On Zoka Quantic a transaction publishes, for its inputs, **only** their nullifiers and the root of the note-commitment tree they belong to. A Groth16 proof shows that:

1. each spent note is a leaf of the tree under that root;
2. the spender knows the note's secrets;
3. each nullifier is correctly derived from the note;
4. inputs equal outputs plus fee, and every amount is in range.

The nullifier is a hash keyed with a nullifier key `nk` that is derived from the wallet seed by a pseudorandom function and never published as a curve point. Breaking every public key on the chain therefore does not let anyone compute nullifiers and match them to notes.

The anonymity set of a spend becomes every note in the tree, not a ring of a few.

Costs: every spend carries a proof. Proving on a 32-bit Android phone takes on the order of fifteen seconds with today's circuit; the new circuit will be benchmarked on the same phone. The circuit changes require the trusted-setup ceremony of section 7.3 to run after it is final.

4.7 Snapshot Signatures — Planned

A fresh node can bootstrap from a signed state snapshot instead of replaying the chain. Those snapshots will carry both an Ed25519 and an ML-DSA-65 signature, and a node accepts a snapshot only if both verify. The snapshot is then checked against the chain it describes, so the signature decides only whom to download from, never what is true.

---

5. What a Quantum Computer Could Do

An adversary with a large quantum computer, reading the archived chain, could…	Zenith	Zoka Quantic (design)
Learn who received an output, given the recipient's address	Yes	No, unless ML-KEM also breaks
Read the amount of an output, given the recipient's address	Yes	No, unless ML-KEM also breaks
Link each spend to the note it consumed	Yes — and anyone can, today, without one	No
Forge proofs or create coins, once the machine exists	Yes	Yes; answered by a protocol upgrade
Spend others' outputs, once the machine exists	Yes	Yes; answered by a protocol upgrade

The version 1.0 whitepaper stated that past privacy would survive a quantum break. For Zenith that is not true, and this table replaces it.

---

6. Transaction and Validation Pipeline — Zoka Quantic draft

A private transaction carries:

- a format version;
- input nullifiers and the note-commitment-tree root;
- outputs: `R`, ML-KEM ciphertext, one-time key `P`, view tag, commitment, range proof and encrypted amount opening;
- a fee commitment with its range proof;
- one Groth16 proof.

Every full node checks, in order: structure and version; that no nullifier has been seen; that the tree root is one the chain has produced; every range proof; and the Groth16 proof. A transaction that fails any step is rejected at the mempool and never reaches a block.

---

7. Consensus and Network

7.1 Proof of Work

ZOKA uses a RandomX-compatible proof of work (`zoka-randomx-v1`), designed for ordinary CPUs. The target block time is 120 seconds and coinbase outputs mature after 60 blocks. Difficulty adjusts every block. Finality is probabilistic; there is no validator set and no stake. The initial difficulty of Zoka Quantic is fixed before genesis, so that the first blocks are not produced faster than the schedule in section 8 assumes.

7.2 Transport

Nodes talk over libp2p. There is no central server: the desktop wallet runs its own Tor and can publish an onion seed, so nodes are reachable without open ports. A fresh node can pull a snapshot over iroh, a QUIC transport with hole punching, from any peer that has one.

The Android wallet joins the same mesh through an embedded Tor client, downloads blocks and scans them on the phone. No server learns which notes belong to it.

Public seed servers will be added for faster first contact, in addition to Tor and never instead of it. The network is designed to keep working if every one of them disappears.

7.3 Trusted Setup

Groth16 needs per-circuit parameters. Zenith's parameters were generated by a single operator, which means that operator could in principle forge proofs. Zoka Quantic's parameters will come from a multi-party ceremony run on the final circuits, which is secure as long as one participant destroys their contribution. Every node verifies the parameter bundle against a published digest and refuses to start on a mismatch.

---

8. Token Economics — ZKA on Zoka Quantic

Parameter	Value
Symbol	ZKA
Decimals	6 (1 atom = 0.000001 ZKA)
Maximum supply	23,000,000 ZKA
Team allocation at genesis	5,000,000 ZKA (21.7%), spendable from block 0
Mined supply	18,000,000 ZKA
Initial block reward	12 ZKA
Halving interval	1,051,200 blocks (about 4 years at 120 s)
Tail emission	None

Emission schedule:

Era	Reward	Blocks	Mined in era	Total supply at end of era

—	Genesis	—	—	5,000,000 (team allocation)
0	12 ZKA	1,051,200	12,614,400 ZKA	17,614,400 ZKA
1	6 ZKA	897,600	5,385,600 ZKA	23,000,000 ZKA

Mining ends at block 1,948,800, about 7.4 years after genesis.

On the team allocation. The version 1.0 whitepaper said "no premine, no founder allocation", and Zenith had none. Zoka Quantic has one, and it is stated here plainly: 5,000,000 ZKA to the team, at an address published in the genesis file, spendable from the first block, with no lock-up. Its amount is public in the genesis block, like every block subsidy, so anyone can verify the total supply at any height.

Block rewards are paid to private outputs. The subsidy amount is public so that emission can be audited; the recipient is a stealth output.

---

9. Privacy Guarantees — Zoka Quantic design

9.1 What the Chain Hides

Field	Mechanism
Recipient	Hybrid stealth address (section 4.5)
Amount	Pedersen commitment; opening encrypted under the hybrid secret
Fee	Pedersen commitment
Which note a transaction spends	Membership proven inside the zero-knowledge proof (section 4.6)
Sender	No address, key or note of the sender is published

9.2 What the Chain Necessarily Publishes

Field	Reason
Block height, timestamp, proof of work	Consensus
Transaction hash; that a transaction occurred	Inherent to a public ledger
Nullifiers	Double-spend prevention; not linkable to notes
Note-commitment-tree roots	Anchors for the membership proof
Output one-time keys and ciphertexts	So recipients can find their outputs
Block subsidy and the genesis allocation	So the supply can be audited

---

10. Honest Limits

- **Validity is classical.** A large quantum computer could forge proofs, create coins or spend others' outputs once it exists. The answer is an upgrade to hash-based proofs before that happens; it is a plan, not a shipped feature.
- **The post-quantum libraries are young.** The pure-Rust ML-KEM and ML-DSA implementations we intend to use are pre-1.0 and, to our knowledge, not independently audited. The external review before genesis covers how ZOKA uses them.
- **Everything is bigger.** About one kilobyte more per output, and addresses of roughly 1,700 characters.
- **Scanning is slower on low-end phones.** Numbers will be published after the testnet benchmarks, not before.
- **Zenith's history stays linkable.** Its transaction graph is public (section 1). Moving to Zoka Quantic does not change what Zenith already published.
- **The team allocation is real.** 21.7% of the supply, spendable from block 0.
- **A global passive adversary** watching both ends of every link can correlate timing. Tor reduces this risk; it does not remove it.
- **There is no recovery.** A lost seed is lost funds, and nobody can reverse a transaction.

---

11. Roadmap

In order, without dates:

1. October 2026 security update: Tor 0.4.9.13 and dependency updates. Done.
2. Removal of the P2P Exchange. Done.
3. Post-quantum building blocks and on-device benchmarks.
4. Zenith becomes the Zoka testnet.
5. Hybrid stealth addresses on the testnet.
6. Public seed servers, in addition to Tor.
7. Spending without publishing the spent note.
8. Hybrid snapshot signatures.
9. Before genesis: networking-stack upgrade, emission tuning, multi-party ceremony, external review.
10. Zoka Quantic genesis.

---

12. Reference Code

Source code is hosted on GitHub at github.com/zoka-network and mirrored on GitLab at gitlab.com/zokanetwork: the node (`zokanetwork`), the desktop wallet (`zsilent-core`), the Android wallet (`zkapriv`), the explorer (`zokaexplorer`) and this site (`zokaweb`). Every release ships SHA-256 checksums and the trusted-setup digest, so an operator can verify a binary and its parameters before running anything.

---

13. Closing Position

ZOKA exists because the right to transact in private is the last frontier of individual freedom. Version 1.0 promised the strongest privacy that implementation knew how to ship, with its limits in plain text. Version 2.0 keeps that promise and adds what we learned since: privacy has to be measured against the computers of the future, and against our own mistakes. Both are written down here.

Verify what is true. Reveal nothing else.

— Natosh Zoka, core dev @ ZokaNetwork